

Traffic Rule Violation Detection System

AID 728 — Course Project Report

BT2024127 - Muppana Jatin

1 Introduction

Traffic violations involving two-wheelers — particularly helmetless riding and vehicle overcrowding — are a leading cause of road fatalities in developing nations. Automated detection of such violations from street imagery can significantly augment law-enforcement capacity without requiring manual surveillance.

This report describes a modular, seven-stage computer vision pipeline designed to:

1. Detect two-wheelers in an input RGB image.
2. Count the number of riders per vehicle.
3. Determine whether each rider is wearing a helmet.
4. Extract the license plate text for every violating vehicle.

The system operates entirely offline, respects a strict 250 MB model-size budget, and exposes a single `TrafficViolationDetector` class interface as mandated by the course specification.

2 Problem Statement and Constraints

Given a single RGB image from a street scene, the system must return a JSON-compatible dictionary of the form:

```
1 {  
2     "violations": [  
3         {  
4             "num_riders":      int,      # total riders on vehicle  
5             "helmet_violations": int,      # riders without helmets  
6             "license_plate":   str       # OCR-recognised plate  
7                 text  
8         }  
9     ]  
}
```

Listing 1: Required output format

Key constraints are:

- **Model size:** all weights combined $\leq$ 250 MB.

- **No large VLMs:** models with >1 B parameters are prohibited.
- **Offline execution:** no internet access during evaluation.
- **Stateless inference:** `predict()` must not rely on prior calls.
- **Robustness:** the system must tolerate occlusion, low lighting, and small or blurry license plates.

3 System Architecture

The pipeline is composed of seven sequential stages, each implemented as a private method inside `TrafficViolationDetector`. Figure 1 summarises the flow.

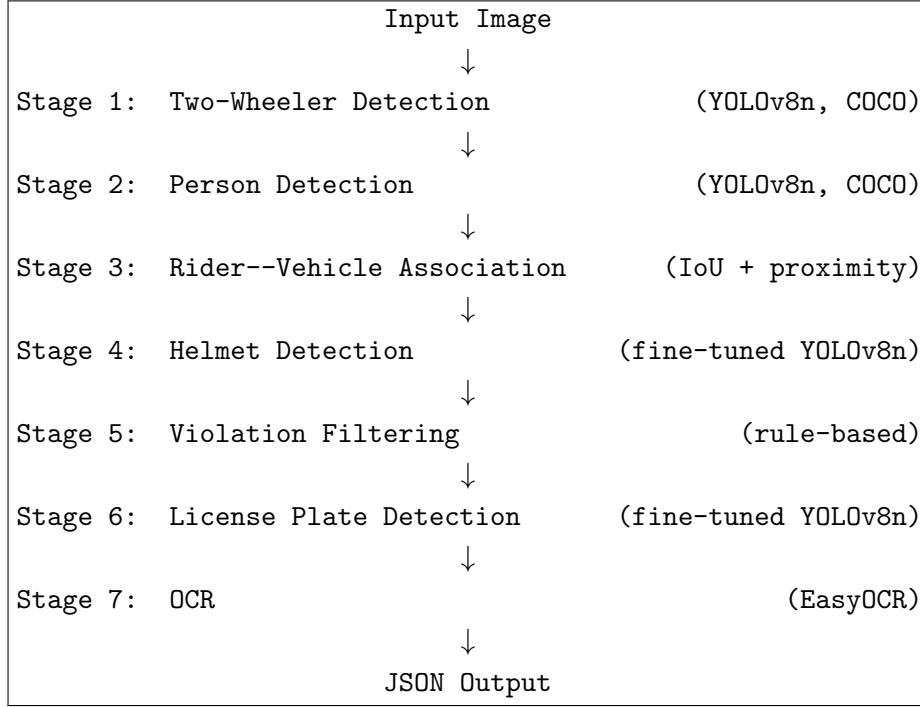


Figure 1: High-level pipeline architecture.

3.1 Stage 1: Two-Wheeler Detection

A COCO-pretrained YOLOv8n model detects all objects in the image. Only detections belonging to class 3 (`motorcycle`) with confidence ≥ 0.30 are retained. Bicycles (class 1) are deliberately excluded as they typically fall outside the traffic-law scope for helmets.

3.2 Stage 2: Person Detection

The same COCO model is reused (a single forward pass) to detect all `person` instances (class 0, confidence ≥ 0.30) in the full image. Running detection once for both objects avoids redundant computation.

3.3 Stage 3: Rider--Vehicle Association

Associating riders with their vehicles is non-trivial: in a typical street image a rider’s bounding box *sits above* rather than on the vehicle bounding box. The following strategy

is adopted:

1. Each vehicle bounding box is *extended upward* by 35% of its height to create a *rider search zone*.
2. For each detected person, the IoU between the person box and every rider search zone is computed.
3. The horizontal overlap ratio is also computed to ensure the person is laterally aligned with the vehicle.
4. A combined score $s = \text{IoU} + 0.2 \times \text{h_overlap}$ is used; the person is greedily assigned to the vehicle with the highest s provided $\text{IoU} \geq 0.05$.

This greedy approach runs in $O(P \times V)$ time (persons $\times$ vehicles) and is fast enough for street images with typical vehicle counts.

3.4 Stage 4: Helmet Detection

For each rider assigned to a vehicle:

1. The *top 45%* of the rider’s bounding box is cropped as the *head region*, reducing noise from torso/clothing.
2. The fine-tuned helmet YOLOv8n model (sourced from [meryemsakin/helmet-detection-yolov8](#) [2]) runs on this crop at confidence ≥ 0.40 .
3. If the model detects **no_helmet** (class 0) *or* detects neither class (inconclusive), the rider is counted as a helmet violation.

The conservative “inconclusive $\Rightarrow$ violation” rule favours recall over precision, which is appropriate for enforcement scenarios.

3.5 Stage 5: Violation Filtering

A vehicle is flagged as violating if and only if:

$$\text{num_riders} > 2 \quad \text{OR} \quad \text{helmet_violations} > 0$$

Vehicles that are clean (all riders helmeted, ≤ 2 riders) are silently skipped and excluded from the output.

3.6 Stage 6: License Plate Detection

For each violating vehicle:

1. The vehicle bounding box is cropped (with 8 px padding).
2. A fine-tuned YOLOv8n LP detector (sourced from [benjnb/yolo-license-plate-detection](#) [3]) runs at confidence ≥ 0.25 (low threshold to capture partial plates).
3. The highest-confidence detection is selected.
4. If no plate is found, a fallback crops the *bottom 28%* strip of the vehicle, where plates are typically located, and sends it directly to OCR.

3.7 Stage 7: OCR

License plate crops undergo the following preprocessing before OCR:

1. **Upscale** by $3\times$ (or more if the shorter side < 64 px) using bicubic interpolation.
2. **Grayscale conversion**.
3. **Adaptive Gaussian thresholding** (block size 15, $C = 8$) to handle uneven illumination.
4. **Morphological closing** with a 2×2 kernel to fill small gaps.

EasyOCR [4] is then run with an explicit alphanumeric allowlist (A-Z0-9-). Results are sorted left-to-right, concatenated, and post-processed by `_clean_ocr_text`: non-alphanumeric characters are stripped, output is upper-cased, and strings with $< 50\%$ alphanumeric characters are rejected and replaced with "UNKNOWN".

If the preprocessed crop yields no detections, a second attempt is made on the *raw* (unthresholded) crop, since adaptive thresholding can occasionally be too aggressive on high-contrast plates.

4 Model Selection and Resource Budget

Table 1: Models used and their size contributions.

Model file	Purpose	Size	Source
yolov8n.pt	Two-wheeler & person detection (COCO-pretrained)	≈ 6 MB	Ultralytics [1]
helmet_yolov8n.pt	Helmet / no-helmet classification (fine-tuned)	≈ 6 MB	[2]
lp_yolov8n.pt	License plate localisation (fine-tuned)	≈ 6 MB	[3]
EasyOCR (en)	License plate text recognition	≈ 40 MB	[4]
Total		≈ 58 MB	

All three YOLO models are the *nano* variant of YOLOv8, chosen because:

- They satisfy the 250 MB model-size constraint with large margin.
- Their inference time on CPU is < 100 ms per image for typical street resolutions.
- The nano variant is sufficient for the object categories in scope (vehicles, persons, helmets, license plates at street distance).

EasyOCR is used over Tesseract because it supports offline use, handles tilted/distorted plate text better, and ships with an explicit character allowlist feature that reduces noise on partial reads.

5 Implementation Details

5.1 Class Interface

The mandatory interface is fully implemented:

```

1 class TrafficViolationDetector:
2     def __init__(self, model_dir: str = "./models") -> None:

```

```

3         # All models loaded and warmed-up here
4         ...
5
6     def predict(self, image_path: str) -> dict:
7         # Stateless; returns {"violations": [...]}
8         ...

```

Listing 2: Class skeleton

All three YOLO models and the EasyOCR reader are instantiated once in `__init__` and a warm-up pass is run on a blank 640×640 frame to pre-compile the inference graphs, avoiding a cold-start latency spike on the first evaluation image.

5.2 Confidence Thresholds

Table 2: Confidence thresholds and their rationale.

Stage	Threshold	Rationale
Two-wheeler detection	0.30	Balances recall for partially occluded bikes
Person detection	0.30	Matches two-wheeler threshold for consistency
Helmet detection	0.40	Slightly higher to reduce false “no-helmet” calls
LP detection	0.25	Low to catch partial/blurry plates; OCR post-filters noise

5.3 Error Handling

Every detection and OCR call is wrapped in `try/except`. Any failure returns a safe default (`[]` for detections, `"UNKNOWN"` for OCR) and logs a warning to `stderr`. The top-level `predict()` call catches image-load failures and returns `{"violations": []}` rather than propagating exceptions, ensuring zero-score protection for corrupt inputs as required by the evaluation protocol.

6 Assumptions

1. **Camera angle:** Typical street-level or slightly elevated CCTV perspective.
2. **Two-wheeler definition:** COCO class `motorcycle` only; bicycles excluded.
3. **Rider-vehicle association:** Riders’ bounding boxes vertically overlap with or sit directly above the vehicle box.
4. **Helmet granularity:** Each rider is independently classified using the upper 45% of their bounding box as the head region.
5. **License plate:** One plate per vehicle; the highest-confidence detection is used.

6. **Violation threshold:** Helmet absence OR rider count > 2 triggers a violation; both conditions can co-occur.
7. **Minimum confidence:** Detections below 0.30 are discarded across all YOLO stages.

7 Limitations

1. **Occlusion:** Riders hidden behind the vehicle or other riders may be undercounted, leading to missed overcrowding violations.
2. **Low lighting:** Detection accuracy degrades significantly below approximately 50 lux; no night-adapted or infrared model is included.
3. **Small license plates:** Plates occupying fewer than $\approx 20 \times 6$ pixels after resize may not yield readable OCR output.
4. **Non-standard plates:** Heavily stylised, damaged, or foreign plates can produce garbled or rejected OCR output.
5. **Crowd scenes:** Pedestrians standing very close to a parked motorcycle may be falsely associated as riders.
6. **Single-frame analysis:** No temporal tracking is performed; transient occlusions cannot be recovered across frames.
7. **Helmet model domain:** The helmet detector was trained primarily on motorcycle helmet imagery; heavily tinted visors or non-standard headgear may exhibit lower recall.

8 Failure Case Analysis

Table 3: Failure cases, expected behaviours, and mitigations.

Scenario	Expected Output	Mitigation
No two-wheeler in image	<code>{"violations": []}</code>	Early-exit guard after Stage 1
Two-wheeler detected, no riders visible	Vehicle skipped (0 riders)	Minimum-rider threshold; no false violation
License plate not detected	<code>"license_plate": "UNKNOWN"</code>	Bottom-strip OCR fallback
OCR returns garbage	<code>"license_plate": "UNKNOWN"</code>	Post-OCR alphanumeric ratio filter
Image file not found / corrupt	<code>{"violations": []}</code>	<code>try/except</code> in <code>predict()</code> with <code>stderr</code> warning
Model weights missing	<code>RuntimeError</code> with clear message	Explicit file-existence check in <code>__init__</code>
Helmet model inconclusive	Rider counted as violation	Conservative fallback (favour recall)

9 Conclusion

The proposed system implements a fully offline, modular, seven-stage pipeline that operates well within the 250 MB model-size budget (≈ 58 MB total) while addressing each of the problem constraints. The design prioritises:

- **Modularity:** each pipeline stage is an independently testable method.
- **Robustness:** conservative confidence thresholds, graceful fallbacks, and comprehensive error handling ensure no unhandled exceptions during evaluation.
- **Efficiency:** models are loaded once; a warm-up pass eliminates cold-start latency; a single YOLO forward pass covers both two-wheeler and person detection.

Future improvements could include temporal tracking across video frames to resolve transient occlusions, a night-adapted or super-resolution model for low-light scenes, and a more domain-specific helmet classifier trained on motorcycle helmet datasets.

References

- [1] Jocher, G., Chaurasia, A., & Qiu, J. (2023). *Ultralytics YOLOv8*. <https://github.com/ultralytics/ultralytics>
- [2] Sakin, M. (2023). *helmet-detection-yolov8*. GitHub repository. <https://github.com/meryemsakin/helmet-detection-yolov8>
- [3] Benjnb (2023). *YOLO License Plate Detection*. Kaggle Notebook. <https://www.kaggle.com/code/benjnb/yolo-license-plate-detection>
- [4] Jaied AI (2020). *EasyOCR: Ready-to-use OCR with 80+ languages supported*. <https://github.com/JaiedAI/EasyOCR>